Lab Assignment : 17.1

NAME:B.Rohith
HT.No:2303A51882
Batch No:02

Task 1: Perform EDA on Penguin dataset(from seaborn)

- Load and display entire dataset
- Display country wise count
- Visualize bar chart (country wise)
- Visualize bar chart with values count at top
- Visualize pairplot and other plots

Prompt : 1. Use CUDA and load and display entire penguin dataset from seaborn

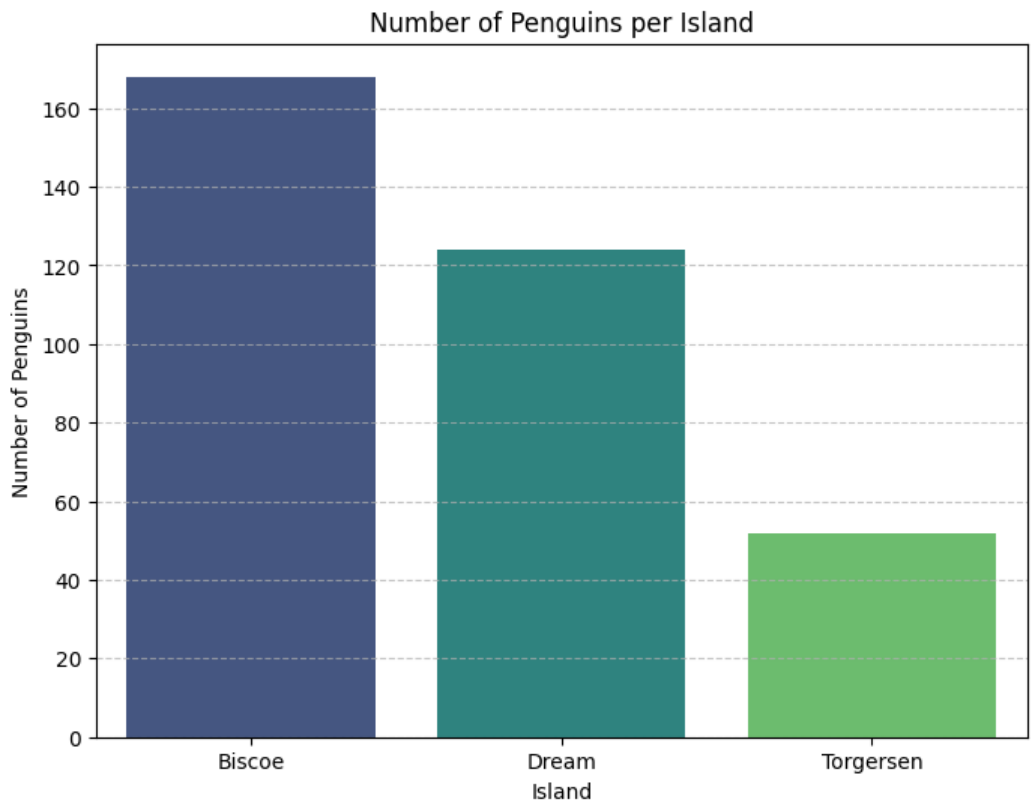
	species	island	bill_length_mm	bill_depth_mm	flipper_length_mm	body_mass_g	se
0	Adelie	Torgersen	39.1	18.7	181.0	3750.0	Mal
1	Adelie	Torgersen	39.5	17.4	186.0	3800.0	Femal
2	Adelie	Torgersen	40.3	18.0	195.0	3250.0	Femal
3	Adelie	Torgersen	NaN	NaN	NaN	NaN	Nal
4	Adelie	Torgersen	36.7	19.3	193.0	3450.0	Femal

Prompt : 2. Display the count of number of penguins per island

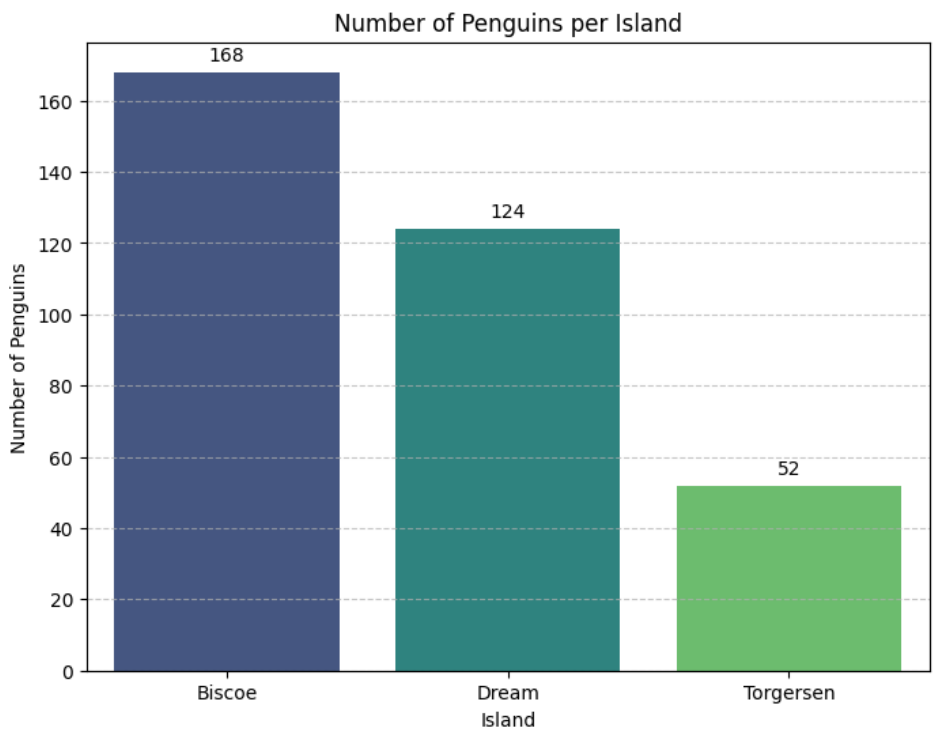
island	count
Biscoe	168
Dream	124
Torgersen	52

dtype: int64

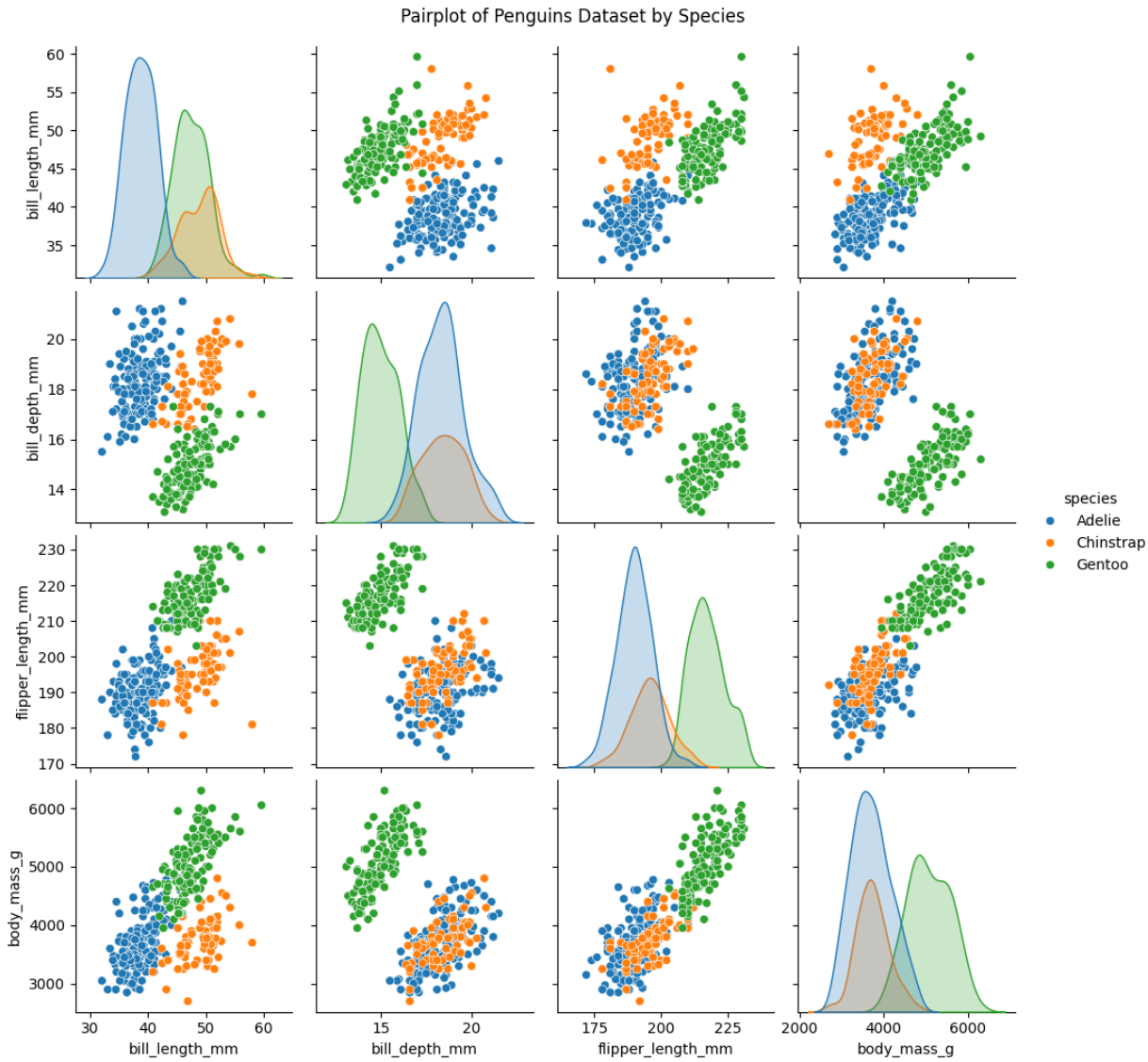
Prompt : 3. visualise the bar chart for island wise



Prompt : 4 Visualize bar chart with values count at top



Prompt : 5 Visualize pairplot and other plots



Task 2: Perform EDA on HealthExp dataset(seaborn)

- Load and display entire dataset
- Display columns and statistical summary (describe)
- Display missing values , and draw bar chart
- Visualize pairplot, regplot and box plot, scatter plot etc

Prompt : 1 Use CUDA and load and display HealthExp dataset from seaborn

1 to 5 of 5 entries Filter

index	Year	Country	Spending_USD	Life_Expectancy
0	1970	Germany	252.311	70.6
1	1970	France	192.143	72.2
2	1970	Great Britain	123.993	71.9
3	1970	Japan	150.437	72.0
4	1970	USA	326.961	70.9

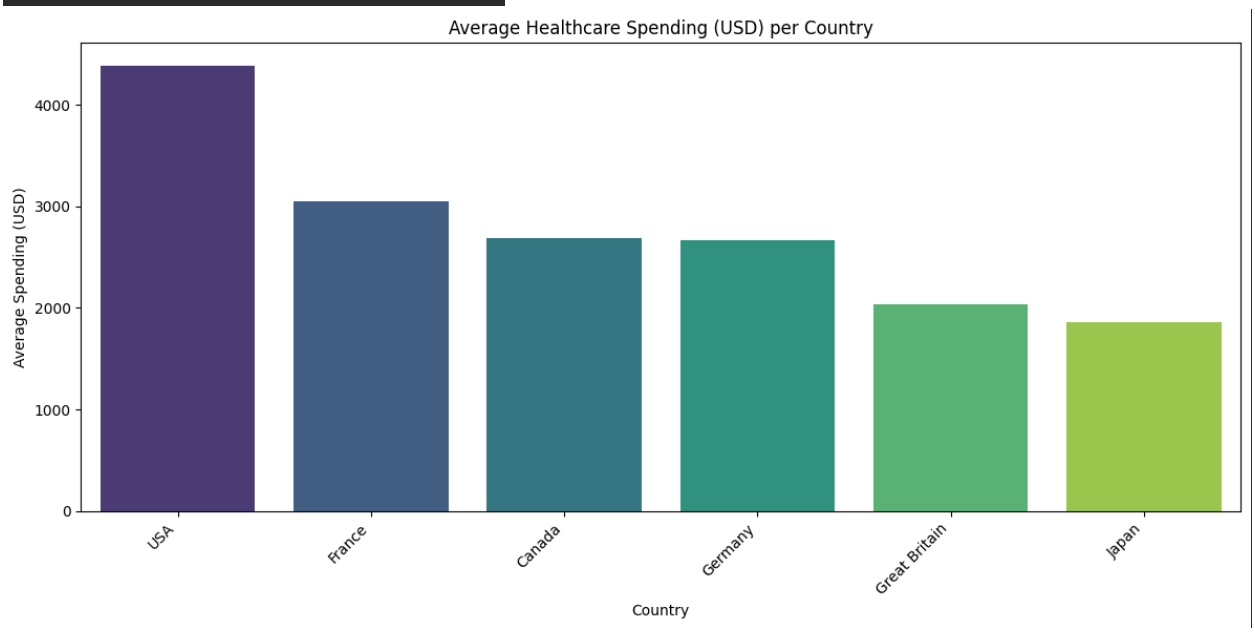
Prompt : 2 Display columns and statistical summary for the given data set

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 274 entries, 0 to 273
Data columns (total 4 columns):
 #   Column           Non-Null Count  Dtype
---  ------           --------------  -----
 0   Year             274 non-null    int64
 1   Country          274 non-null    object
 2   Spending_USD     274 non-null    float64
 3   Life_Expectancy  274 non-null    float64
dtypes: float64(2), int64(1), object(1)
memory usage: 8.7+ KB
None
```

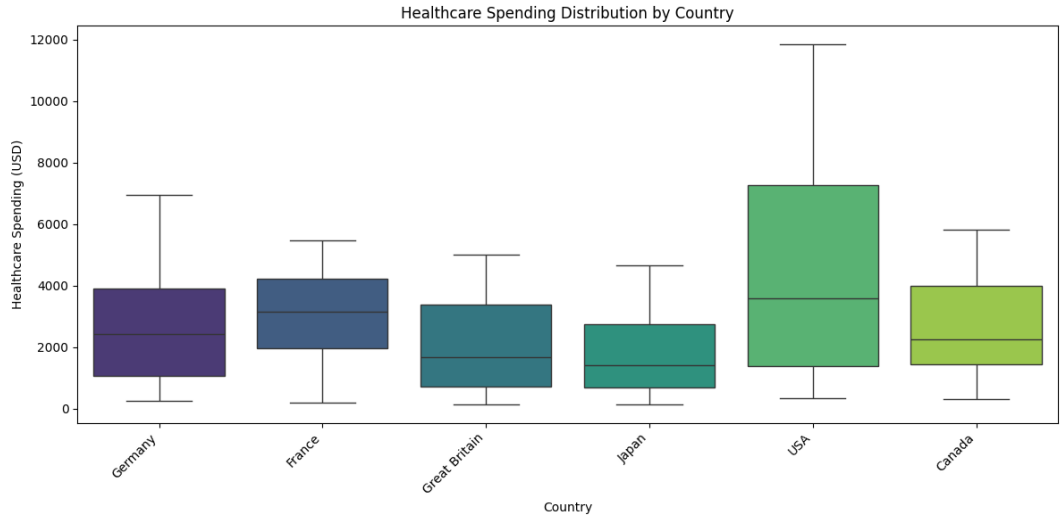
	Year	Spending_USD	Life_Expectancy
count	274.000000	274.000000	274.000000
mean	1996.992701	2789.338905	77.909489
std	14.180933	2194.939785	3.276263
min	1970.000000	123.993000	70.600000
25%	1985.250000	1038.357000	75.525000
50%	1998.000000	2295.578000	78.100000
75%	2009.000000	4055.610000	80.575000
max	2020.000000	11859.179000	84.700000

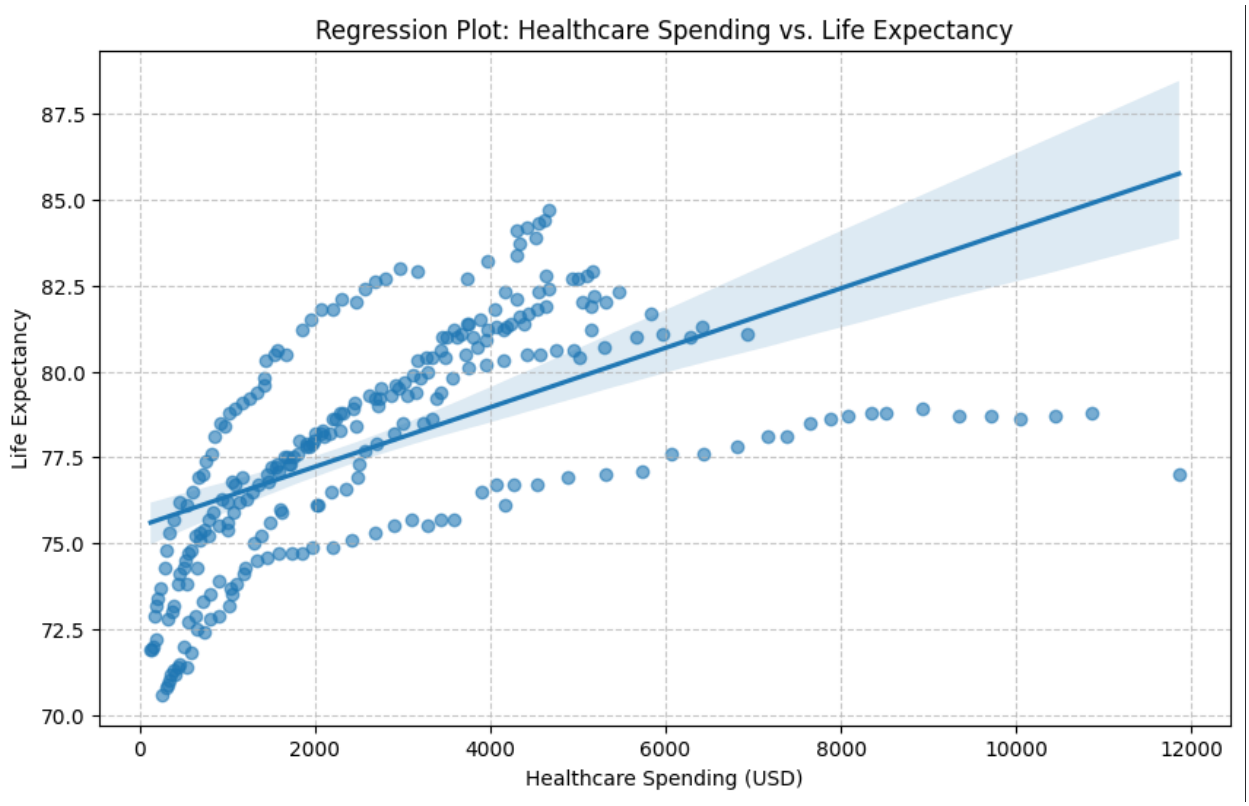
Prompt : 3 Display missing values , and draw bar chart

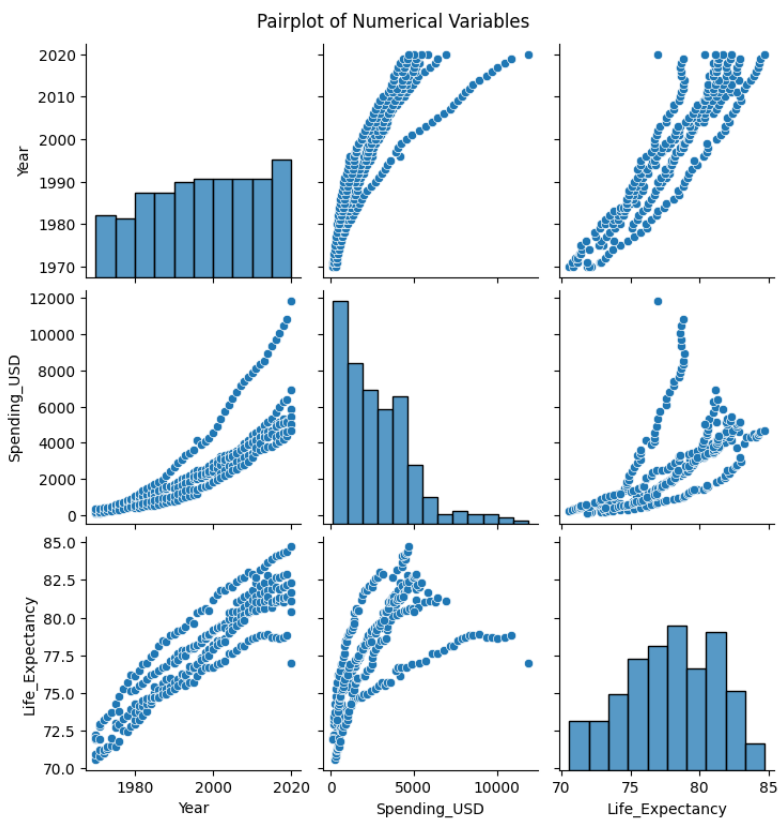
```
••• Missing Values per Column:
                    0
Year                0
Country             0
Spending_USD        0
Life_Expectancy     0

dtype: int64
```

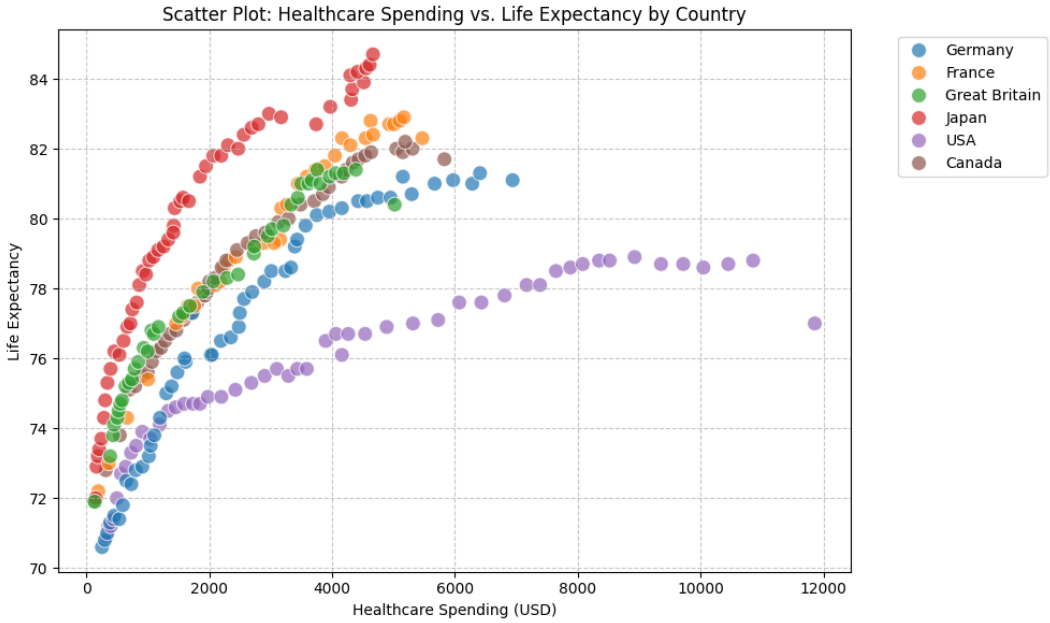


Prompt : 4 Visualize the pairplot, regplot and box plot, scatter plot and etc




Regression Plot: Healthcare Spending vs. Life Expectancy
Life Expectancy
Healthcare Spending (USD)
87.5
85.0
82.5
80.0
77.5
75.0
72.5
70.0
0
2000
4000
6000
8000
10000
12000


Pairplot of Numerical Variables
Year
Spending_USD
Life_Expectancy
2020
2010
2000
1990
1980
1970
12000
10000
8000
6000
4000
2000
0
85.0
82.5
80.0
77.5
75.0
72.5
70.0
1980
2000
2020
0
5000
10000
70
75
80
85



Task 3: Perform following task on given dataset (Train the model)

- Analyze the Planets dataset to understand its structure, features, and missing values.
- Perform data cleaning and preprocessing to handle missing or incorrect data.
- Train the dataset using different machine learning models to make predictions.
- Compare and explain the performance of all trained models with proper reasoning.

Prompt : 1 Use CUDA and load Planets dataset from seaborn and Analyze the Planets dataset to understand its structure, features, and missing values.

```
Missing values per column:
```

	0
method	0
number	0
orbital_period	43
mass	522
distance	227
year	0

dtype: int64

```
Dataset loaded successfully.
First 5 rows of the dataset:
```

	method	number	orbital_period	mass	distance	year
0	Radial Velocity	1	269.300	7.10	77.40	2006
1	Radial Velocity	1	874.774	2.21	56.95	2008
2	Radial Velocity	1	763.000	2.60	19.84	2011
3	Radial Velocity	1	326.030	19.40	110.62	2007
4	Radial Velocity	1	516.220	10.50	119.47	2009

```
Dataset Structure (info):
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1035 entries, 0 to 1034
Data columns (total 6 columns):
 #   Column          Non-Null Count  Dtype
---  ------          --------------  -----
 0   method          1035 non-null   object
 1   number          1035 non-null   int64
 2   orbital_period  992 non-null    float64
 3   mass            513 non-null    float64
 4   distance        808 non-null    float64
 5   year            1035 non-null   int64
dtypes: float64(3), int64(2), object(1)
memory usage: 48.6+ KB
```

```
Missing values before imputation:
```

	0
method	0
number	0
orbital_period	43
mass	522
distance	227
year	0

dtype: int64

Prompt : 2 Perform data cleaning and preprocessing to handle missing or incorrect data

```
Median 'orbital_period': 39.9795
Median 'mass': 1.26
Median 'distance': 55.25

Missing values imputed with median.
```

```
Missing values after imputation:
```

	0
method	0
number	0
orbital_period	0
mass	0
distance	0
year	0

dtype: int64

Prompt : 3 Train the dataset using different machine learning models to make predictions.

```
Shape of X_train: (828, 4)
Shape of X_test: (207, 4)
Shape of y_train: (828,)
Shape of y_test: (207,)
```

```
--- Training Linear Regression ---
Linear Regression MAE: 1.5065
Linear Regression R2 Score: 0.0424

--- Training Random Forest Regressor ---
Random Forest Regressor MAE: 1.3336
Random Forest Regressor R2 Score: 0.0525

--- Training Gradient Boosting Regressor ---
Gradient Boosting Regressor MAE: 1.2310
Gradient Boosting Regressor R2 Score: 0.1796

--- Model Comparison ---
Linear Regression: MAE = 1.5065, R2 = 0.0424
Random Forest Regressor: MAE = 1.3336, R2 = 0.0525
Gradient Boosting Regressor: MAE = 1.2310, R2 = 0.1796
```

Prompt : 4 Compare and explain the performance of all trained models with proper reasoning

Task 4: Perform following task on given dataset (Train the model)

- Load the MNIST dataset from PyTorch.
- Display basic information and visualize sample input images with their class labels.
- Perform data preprocessing
- Design a EfficientNet model for image classification.
- Train the CNN model (10 epochs)
- Evaluate model performance using accuracy and loss metrics.
- Visualize the training history (accuracy and loss curves).

```
--- Model Comparison Results ---
Linear Regression: MAE = 1.5065, R2 = 0.0424
Random Forest Regressor: MAE = 1.3336, R2 = 0.0525
Gradient Boosting Regressor: MAE = 1.2310, R2 = 0.1796
```

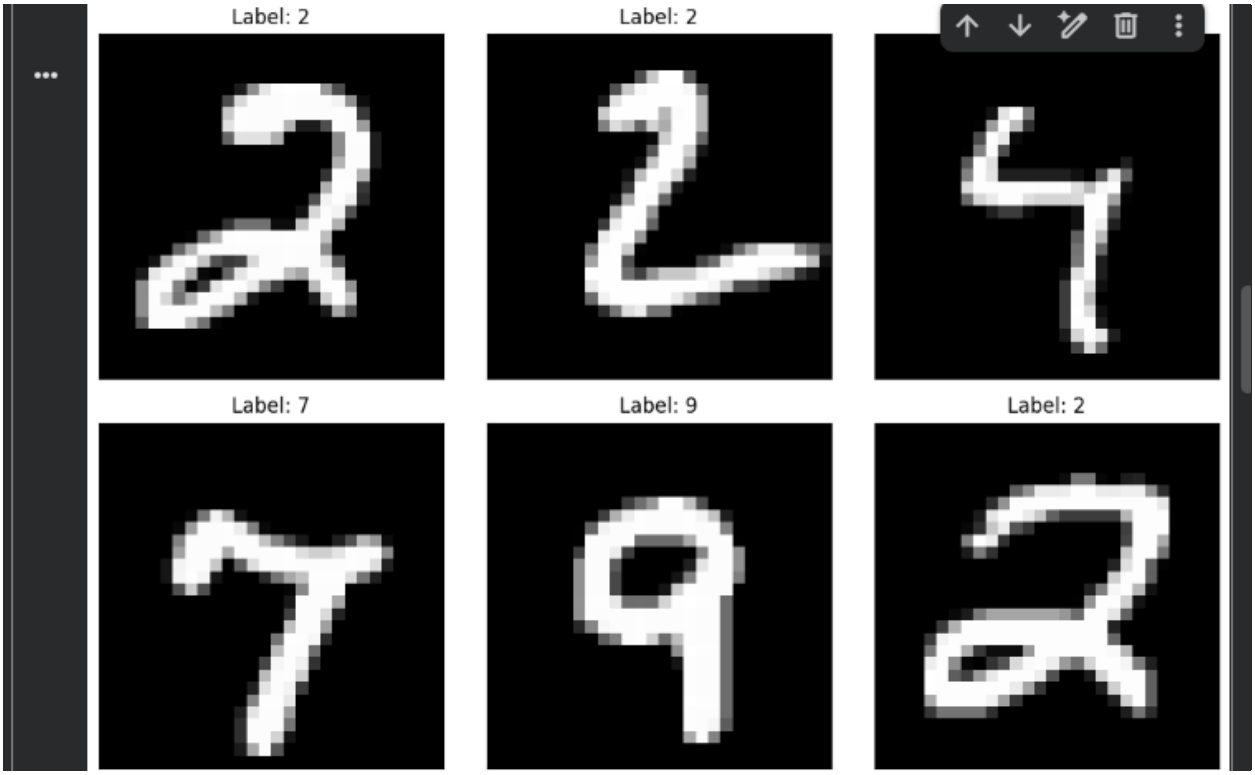
Prompt : 1 Use Cuda and Load the MNIST dataset from PyTorch.

```
100%|          | 9.91M/9.91M [00:01<00:00, 5.08MB/s]
100%|          | 28.9k/28.9k [00:00<00:00, 133kB/s]
100%|          | 1.65M/1.65M [00:01<00:00, 1.25MB/s]
100%|          | 4.54k/4.54k [00:00<00:00, 8.19MB/s]
```

```
Number of training batches: 938
Number of test batches: 157
Shape of a batch of images: torch.Size([64, 1, 28, 28])
Shape of a batch of labels: torch.Size([64])
Shape of images after moving to device: torch.Size([64, 1, 28, 28])
```

Prompt : 2 Display basic information and visualize sample input images with their class labels.

```
Number of training samples: 60000
Number of test samples: 10000
Image shape: 28x28
Number of classes: 10
Class labels: ['0 - zero', '1 - one', '2 - two', '3 - three', '4 - four', '5 - five', '6 -
```



Prompt : 4 create a efficient net model for image classification

```
Starting training...
Epoch [1/5], Loss: 0.2677, Accuracy: 91.95%
Epoch [2/5], Loss: 0.0850, Accuracy: 97.66%
Epoch [3/5], Loss: 0.0631, Accuracy: 98.27%
Epoch [4/5], Loss: 0.0582, Accuracy: 98.43%
Epoch [5/5], Loss: 0.0469, Accuracy: 98.76%
Training complete.
```

```
Accuracy of the model on the 10000 test images: 98.74%
```

Prompt : 5 train the cnn model upto 10 epochs

```
Starting training...
Epoch [1/10], Loss: 0.0541, Accuracy: 98.59%
Epoch [2/10], Loss: 0.0525, Accuracy: 98.64%
Epoch [3/10], Loss: 0.0403, Accuracy: 98.90%
Epoch [4/10], Loss: 0.0348, Accuracy: 99.08%
Epoch [5/10], Loss: 0.0392, Accuracy: 98.94%
Epoch [6/10], Loss: 0.0330, Accuracy: 99.17%
Epoch [7/10], Loss: 0.0207, Accuracy: 99.37%
Epoch [8/10], Loss: 0.0240, Accuracy: 99.39%
Epoch [9/10], Loss: 0.0264, Accuracy: 99.27%
Epoch [10/10], Loss: 0.0233, Accuracy: 99.36%
Training complete.
```

```
••• Accuracy of the model on the 10000 test images: 99.26%
```

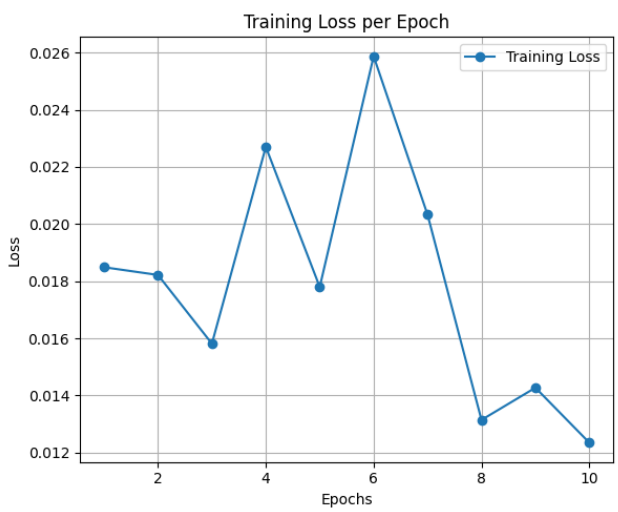
Prompt : 6 **Evaluate the model performance using accuracy and loss metrics.**

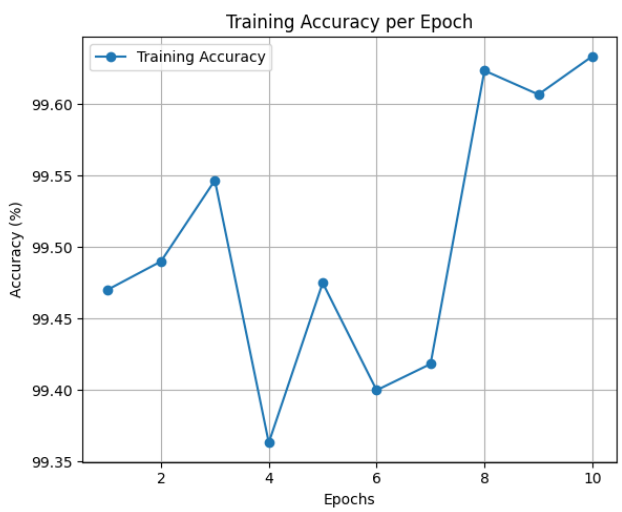
```
••• Accuracy of the model on the 10000 test images: 99.26%
    Average test loss: 0.0224
```

Prompt : 7 Visualize the training history (accuracy and loss curves)

```
••• Starting training...
    Epoch [1/10], Loss: 0.0185, Accuracy: 99.47%
    Epoch [2/10], Loss: 0.0182, Accuracy: 99.49%
    Epoch [3/10], Loss: 0.0158, Accuracy: 99.55%
    Epoch [4/10], Loss: 0.0227, Accuracy: 99.36%
    Epoch [5/10], Loss: 0.0178, Accuracy: 99.47%
    Epoch [6/10], Loss: 0.0259, Accuracy: 99.40%
    Epoch [7/10], Loss: 0.0203, Accuracy: 99.42%
    Epoch [8/10], Loss: 0.0131, Accuracy: 99.62%
    Epoch [9/10], Loss: 0.0143, Accuracy: 99.61%
    Epoch [10/10], Loss: 0.0123, Accuracy: 99.63%
    Training complete.
```

```
••• Accuracy of the model on the 10000 test images: 99.26%
    Average test loss: 0.0224
```





Task 5 – Customer Data Cleaning

Task:

Use AI to generate a Python script for cleaning a **customer dataset**.

Instructions:

- Handle missing values in columns (age, income, city).
- Remove duplicate records.
- Standardize text values (e.g., city names in lowercase).
- Encode categorical variables (gender, city).

Sample Code:

```
import pandas as pd
data = pd.read_csv("shopping_trends.csv")
print(data.head())
```

Dataset link:

https://www.kaggle.com/datasets/iamsouravbanerjee/customer-shopping-trends-dataset?select=shopping_trends.csv

Expected Output:

- A cleaned Pandas data frame ready for analysis.

Prompt : 1 Handle the missing values in the columns (age, income, city)

```
# Check for missing values
print(data.isnull().sum())
```

```
Customer ID                 0
Age                         0
Gender                      0
Item Purchased              0
Category                    0
Purchase Amount (USD)       0
Location                    0
Size                        0
Color                       0
Season                      0
Review Rating               0
Subscription Status         0
Payment Method              0
Shipping Type               0
Discount Applied            0
Promo Code Used             0
Previous Purchases          0
Preferred Payment Method    0
Frequency of Purchases      0
dtype: int64
```

```
No missing values found in 'Age'.
No missing values found in 'Purchase Amount (USD)'.
No missing values found in 'Review Rating'.
No missing values found in 'Gender'.
No missing values found in 'Location'.
No missing values found in 'Category'.
```

Prompt : 2 Remove duplicate records for the given data set

```
Number of duplicate rows found: 0
No duplicate rows to remove.
New DataFrame shape after removing duplicates: (3900, 19)
```

Prompt : 3 Standardize text values (e.g., city names in lowercase).

```
data['Location'] = data['Location'].str.lower()
print(data.head())
```

```
   Customer ID  Age Gender Item Purchased  Category  Purchase Amount (USD)  \
0            1   55   Male         Blouse  Clothing                     53
1            2   19   Male        Sweater  Clothing                     64
2            3   50   Male          Jeans  Clothing                     73
3            4   21   Male        Sandals  Footwear                     90
4            5   45   Male         Blouse  Clothing                     49

        Location Size      Color  Season  Review Rating Subscription Status  \
0       kentucky    L       Gray  Winter            3.1                 Yes
1          maine    L     Maroon  Winter            3.1                 Yes
2  massachusetts    S     Maroon  Spring            3.1                 Yes
3   rhode island    M     Maroon  Spring            3.5                 Yes
4         oregon    M  Turquoise  Spring            2.7                 Yes

  Payment Method  Shipping Type Discount Applied Promo Code Used  \
0    Credit Card        Express              Yes             Yes
1   Bank Transfer        Express              Yes             Yes
2           Cash  Free Shipping              Yes             Yes
3         PayPal   Next Day Air              Yes             Yes
4           Cash  Free Shipping              Yes             Yes
```

Prompt : 4 Encode categorical variables (gender, city).

```python
# One-Hot Encode 'Gender' and 'Location' columns
data = pd.get_dummies(data, columns=['Gender', 'Location'], drop_first=True)

# Display the first few rows of the updated DataFrame to show the new encoded columns
print(data.head())
```

```
   Customer ID  Age Item Purchased  Category  Purchase Amount (USD) Size  \
0            1   55         Blouse  Clothing                     53    L
1            2   19        Sweater  Clothing                     64    L
2            3   50          Jeans  Clothing                     73    S
3            4   21        Sandals  Footwear                     90    M
4            5   45         Blouse  Clothing                     49    M

        Color  Season  Review Rating Subscription Status  ...  \
0        Gray  Winter            3.1                 Yes  ...
1      Maroon  Winter            3.1                 Yes  ...
2      Maroon  Spring            3.1                 Yes  ...
3      Maroon  Spring            3.5                 Yes  ...
4  Turquoise  Spring            2.7                 Yes  ...

   Location_south dakota Location_tennessee Location_texas Location_utah  \
0                  False              False         False        False
1                  False              False         False        False
2                  False              False         False        False
3                  False              False         False        False
4                  False              False         False        False

   Location_vermont Location_virginia Location_washington  \
0             False             False               False
1             False             False               False
2             False             False               False
3             False             False               False
4             False             False               False

   Location_west virginia  Location_wisconsin  Location_wyoming
0                   False               False             False
1                   False               False             False
2                   False               False             False
3                   False               False             False
4                   False               False             False

[5 rows x 67 columns]
```

Task 6 – Sales Data Preprocessing

Task:

Preprocess **sales transaction data** using AI assistance.

Instructions:

- Convert date columns into datetime format.
- Create new features (month, year, day of week).
- Normalize sales amount column.
- Detect and handle outliers in dataset.

Sample Code:

```
import pandas as pd
data = pd.read_csv("sales_data.csv")
print(data.head())
```

Dataset link:

https://www.kaggle.com/datasets/vinothkannaece/sales-dataset?select=sales_data.csv

Expected Output:

- A transformed dataset with time-based features and normalized values.

Prompt : 1 Convert date columns into datetime format for given data set

```
df['Sale_Date'] = pd.to_datetime(df['Sale_Date'], errors='coerce')

print("DataFrame Info after date conversion:")
df.info()
```

```
DataFrame Info after date conversion:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1000 entries, 0 to 999
Data columns (total 14 columns):
 #   Column                Non-Null Count  Dtype
---  ------                --------------  -----
 0   Product_ID            1000 non-null   int64
 1   Sale_Date             1000 non-null   datetime64[ns]
 2   Sales_Rep             1000 non-null   object
 3   Region                1000 non-null   object
 4   Sales_Amount          1000 non-null   float64
 5   Quantity_Sold         1000 non-null   int64
 6   Product_Category      1000 non-null   object
 7   Unit_Cost             1000 non-null   float64
 8   Unit_Price            1000 non-null   float64
 9   Customer_Type         1000 non-null   object
 10  Discount              1000 non-null   float64
 11  Payment_Method        1000 non-null   object
 12  Sales_Channel         1000 non-null   object
 13  Region_and_Sales_Rep  1000 non-null   object
dtypes: datetime64[ns](1), float64(4), int64(2), object(7)
memory usage: 109.5+ KB
```

Prompt : 2 Create new features (month, year, day of week) for given data set

```
DataFrame with new date features:
```

	Product_ID	Sale_Date	Sales_Rep	Region	Sales_Amount	Quantity_Sold	Product_Category
0	1052	2023-02-03	Bob	North	5053.97	18	Furniture
1	1093	2023-04-21	Bob	West	4384.02	17	Furniture
2	1015	2023-09-21	David	South	4631.23	30	Food
3	1072	2023-08-24	Bob	South	2167.94	39	Clothing
4	1061	2023-03-24	Charlie	East	3750.20	13	Electronics

Prompt : 3 Normalize the sales amount column.

```
DataFrame with normalized sales amount:
```

	Product_ID	Sale_Date	Sales_Rep	Region	Sales_Amount	Quantity_Sold	Product_Category
0	1052	2023-02-03	Bob	North	5053.97	18	Furniture
1	1093	2023-04-21	Bob	West	4384.02	17	Furniture
2	1015	2023-09-21	David	South	4631.23	30	Food
3	1072	2023-08-24	Bob	South	2167.94	39	Clothing
4	1061	2023-03-24	Charlie	East	3750.20	13	Electronics

Prompt : 4 Detect and handle the outliers in dataset..

```
Q1 (25th percentile): 2550.30
Q3 (75th percentile): 7507.45
IQR: 4957.15
Lower bound for outliers: -4885.42
Upper bound for outliers: 14943.17
Number of outliers in 'Sales_Amount': 0

DataFrame head with original and capped Sales_Amount:
```

	Sales_Amount	Sales_Amount_Capped
0	5053.97	5053.97
1	4384.02	4384.02
2	4631.23	4631.23
3	2167.94	2167.94
4	3750.20	3750.20

Task 7 – Healthcare Data Preparation

Task:

Clean and preprocess a **healthcare dataset** using AI scripts.

Instructions:

- Handle missing values in blood_pressure, cholesterol.
- Encode categorical features
- Scale numerical features (min-max or standard scaling).
- Split dataset into training and testing sets.

Dataset link :

https://www.kaggle.com/datasets/abdallaahmed77/healthcare-risk-factors-dataset

Expected Output:

- A preprocessed dataset ready for machine learning models.

Prompt : 1 Handle the missing values in blood_pressure, cholesterol.

```
Original non-null counts for Blood Pressure and Cholesterol:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 30000 entries, 0 to 29999
Data columns (total 2 columns):
 #   Column          Non-Null Count  Dtype
---  ------          --------------  -----
 0   Blood Pressure  30000 non-null  float64
 1   Cholesterol     30000 non-null  float64
dtypes: float64(2)
memory usage: 468.9 KB
None

Non-null counts after median imputation:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 30000 entries, 0 to 29999
Data columns (total 2 columns):
 #   Column          Non-Null Count  Dtype
---  ------          --------------  -----
 0   Blood Pressure  30000 non-null  float64
 1   Cholesterol     30000 non-null  float64
dtypes: float64(2)
memory usage: 468.9 KB
None
```

Prompt : 2 . Encode the categorical features

```
Categorical columns identified: ['Gender', 'Medical Condition', 'random_notes']

DataFrame after one-hot encoding (first 5 rows):
```

	Age	Glucose	Blood Pressure	BMI	Oxygen Saturation	LengthOfStay	Cholesterol	Triglycerides	Hb/
0	46.0	137.04	135.27	28.90	96.04	6	231.88	210.56	7
1	22.0	71.58	113.27	26.29	97.54	2	165.57	129.41	4
2	50.0	95.24	138.32	22.53	90.31	2	214.94	165.35	5
3	57.0	NaN	130.53	38.47	96.60	5	197.71	182.13	6
4	66.0	95.15	178.17	31.12	94.90	4	259.53	115.85	5

5 rows × 27 columns

Prompt : 3 . Scale the numerical features (min-max or standard scaling).

```
Numerical columns identified for scaling: ['Age', 'Glucose', 'Blood Pressure', 'BMI', 'Oxy

DataFrame after standard scaling (first 5 rows):
```

	Age	Glucose	Blood Pressure	BMI	Oxygen Saturation	LengthOfStay	Cholesterol	Triglyce
0	-0.539449	0.322729	-0.250163	0.073986	0.290409	0.574279	0.562248	0.6
1	-2.041956	-1.251732	-1.381416	-0.381614	0.691893	-0.874214	-1.416021	-0.9
2	-0.289031	-0.682656	-0.093330	-1.037957	-1.243260	-0.874214	0.056866	-0.2
3	0.149200	NaN	-0.493897	1.744520	0.440296	0.212156	-0.457168	0.1
4	0.712640	-0.684820	1.955780	0.461508	-0.014719	-0.149967	1.387149	-1.2

5 rows × 27 columns

Prompt : 4 . Split the dataset into training and testing sets using train_test_split

```
Missing values in df_scaled before final imputation:
Age        4500
Glucose    4500
dtype: int64

Missing values in df_scaled after final imputation:
Series([], dtype: int64)
```

```
Shapes of the resulting datasets:
X_train shape: (24000, 25)
X_test shape: (6000, 25)
y_train shape: (24000, 2)
y_test shape: (6000, 2)
```

Task 8 – Employee Salary Data Preprocessing

Task:

Clean and preprocess an employee salary dataset using AI scripts.

Instructions:

- Handle missing values
- Detect and remove salary outliers.
- Standardize department names (lowercase).
- Apply label encoding to job location, department.

Dataset link:
https://www.kaggle.com/code/vishantmathur/employee-salary

Expected Output:

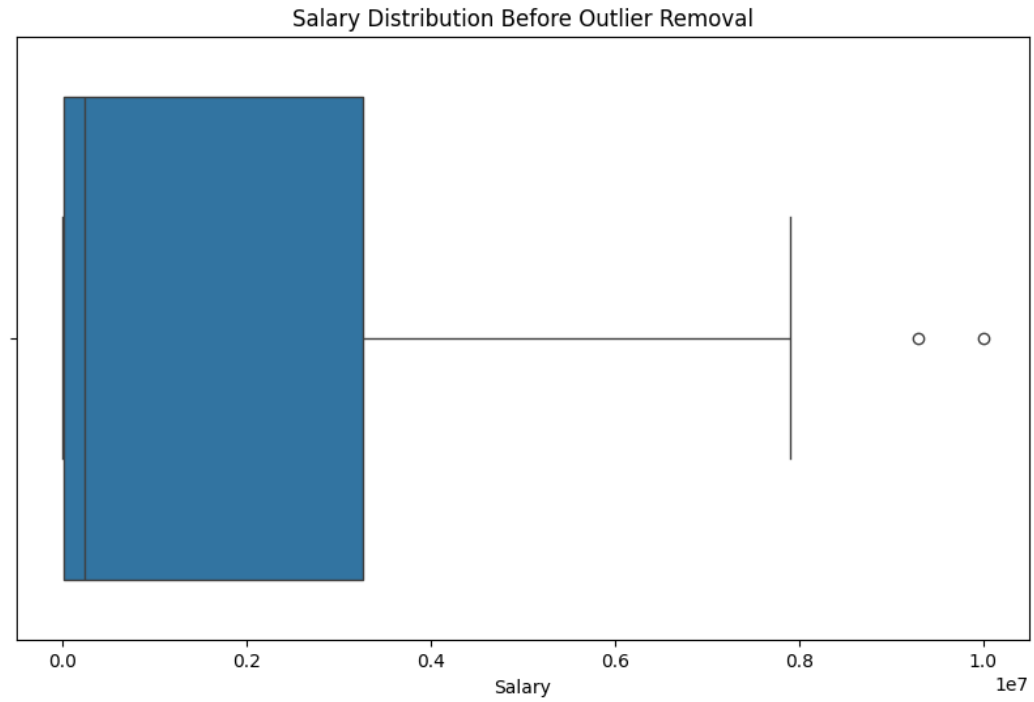
- A structured dataset ready for HR analytics.

Prompt : 1 Handle the missing values

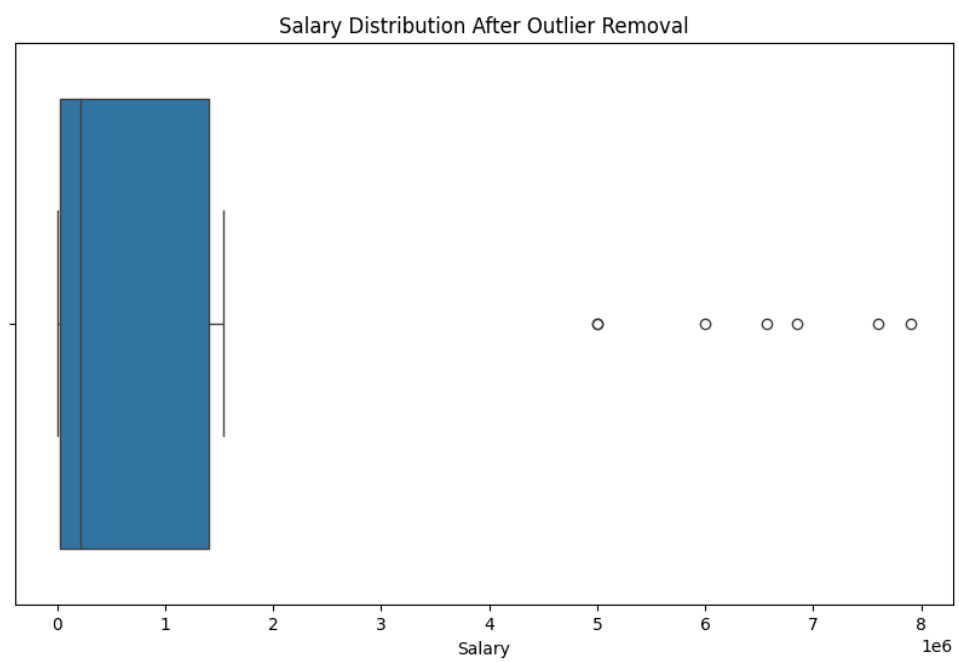
```
Missing values before handling:
                    0
ID                  0
Experience_Years    0
Age                 0
Gender              0
Salary              0
dtype: int64
```

Prompt : 2 give me the code for Detect and remove salary outliers.

```
Shape of DataFrame before outlier removal: (35, 5)
```




```
Shape of DataFrame after outlier removal: (33, 5)
```



Prompt : 3 standardize the department names (lowercase).

	ID	Experience_Years	Age	Gender	Salary
0	1	5	28	female	250000
1	2	1	21	male	50000
2	3	3	23	female	170000
3	4	2	22	male	25000
4	5	1	17	male	10000

Task 9 –Loan Approval Dataset Cleaning

Use AI to generate preprocessing code for a loan dataset.

Instructions:

- Handle missing values in loan_amount, credit_score.
- Encode loan_status (Approved/Rejected).
- Scale numerical features using standard scaling.
- Split the dataset into training and testing sets.

Sample Code:

```
import pandas as pd
data = pd.read_csv("loadapproval.csv")
print(data.head())
```

Dataset link:

https://www.kaggle.com/datasets/amineipad/loan-approval-dataset

Expected Output:

- A machine-learning-ready loan dataset.

Prompt : 1 Handle the missing values in loan_amount, credit_score

```
Missing values before handling:
loan_amount     0
credit_score    0
dtype: int64
```

Prompt : 2 Encode the loan_status (Approved/Rejected).

	applicant_id	age	gender	marital_status	annual_income	loan_amount	credit_score	nu
0	1	59	Male	Divorced	100073	7169	793	
1	2	49	Male	Married	112197	23556	789	
2	3	35	Male	Divorced	84429	27052	372	
3	4	63	Female	Single	124195	11313	808	
4	5	28	Female	Married	81627	13315	689	

Prompt : 3 Scale the numerical features using standard scaling.

```
DataFrame after standard scaling:
```

	applicant_id	age	gender	marital_status	annual_income	loan_amount	credit_scor
0	1	1.307840	Male	Divorced	0.482301	-1.566427	1.39162
1	2	0.514489	Male	Married	0.805362	-0.287825	1.36595
2	3	-0.596204	Male	Divorced	0.065444	-0.015048	-1.30980
3	4	1.625181	Female	Single	1.125066	-1.243090	1.48787
4	5	-1.151550	Female	Married	-0.009219	-1.086883	0.72428

Prompt : 4 Split the data into training and testing sets using train_test_split

```
X_train shape: (800, 6)
X_test shape: (200, 6)
y_train shape: (800,)
y_test shape: (200,)
```

Task 10 – Social Media Profiles

Preprocess a social media engagement dataset using AI assistance.

Instructions:

• Remove duplicate user profiles.

- Handle missing values
- Encode categorical columns

Dataset link:
https://www.kaggle.com/datasets/medaxone/top-100-social-media-profiles

Expected Output:

- A cleaned dataset for engagement prediction.

Prompt : 1 Remove the duplicate user profiles

```
DataFrame shape before dropping duplicates: (500, 5)
DataFrame shape after dropping duplicates: (452, 5)
```

	N	PROFILE	FOLLOWERS	POSTS	Platform
0	1	Instagram	539446645.0	7202.0	Instagram
1	2	Cristiano Ronaldo	473864939.0	3338.0	Instagram
2	3	Kylie <U+0001F90D>	364542529.0	6935.0	Instagram
3	4	Leo Messi	355790796.0	890.0	Instagram
4	5	Selena Gomez	341579063.0	1828.0	Instagram

Prompt : 2 Handle the missing values

```
Missing values in 'POSTS' after filling: 0
```

	N	PROFILE	FOLLOWERS	POSTS	Platform
0	1	Instagram	539446645.0	7202.0	Instagram
1	2	Cristiano Ronaldo	473864939.0	3338.0	Instagram
2	3	Kylie <U+0001F90D>	364542529.0	6935.0	Instagram
3	4	Leo Messi	355790796.0	890.0	Instagram
4	5	Selena Gomez	341579063.0	1828.0	Instagram

```
Missing values before handling:
```

	0
N	0
PROFILE	0
FOLLOWERS	0
POSTS	97
Platform	0

dtype: int64

Prompt : 3 Encode the ategorical columns

```
Categorical columns identified: ['PROFILE', 'Platform']
```

	N	PROFILE	FOLLOWERS	POSTS	Platform_TikTok	Platform_Twitch	Platform_Twitte
0	1	Instagram	539446645.0	7202.0	False	False	Fals
1	2	Cristiano Ronaldo	473864939.0	3338.0	False	False	Fals
2	3	Kylie <U+0001F90D>	364542529.0	6935.0	False	False	Fals
3	4	Leo Messi	355790796.0	890.0	False	False	Fals
4	5	Selena Gomez	341579063.0	1828.0	False	False	Fals

Task 11 – Weather Dataset Feature Engineering

Task:

Use AI to preprocess a weather dataset for forecasting.

Instructions:

- Convert date column into datetime format.
- Handle missing values in temperature, humidity.
- Create new features (season, week_number).
- Normalize rainfall values.

Dataset link:

https://www.kaggle.com/code/alsamaualalhinai/weather-dataset/input

Expected Output:

- A processed dataset ready for forecasting models.

Prompt : 1 Convert the date column into datetime format.

	Temperature	Humidity	Wind_Speed	Cloud_Cover	Pressure	Rain
0	23.720338	89.592641	7.335604	50.501694	1970-01-01 00:00:00.000001032	rain
1	27.879734	46.489704	5.952484	4.990053	1970-01-01 00:00:00.000000992	no rain
2	25.069084	83.072843	1.371992	14.855784	1970-01-01 00:00:00.000001007	no rain
3	23.622080	74.367758	7.050551	67.255282	1970-01-01 00:00:00.000000982	rain
4	20.591370	96.858822	4.643921	47.676444	1970-01-01 00:00:00.000000980	no rain

Prompt : 2 Handle the missing values in temperature, humidity.

```
Missing values before handling:
Temperature    0
Humidity       0
dtype: int64

Missing values after handling:
Temperature    0
Humidity       0
dtype: int64
```

Prompt : 3 Normalize the rainfall values.

	Rain	Rain_Encoded
0	rain	1
1	no rain	0
2	no rain	0
3	rain	1
4	no rain	0

```
object
['rain' 'no rain']
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 2500 entries, 0 to 2499
Data columns (total 7 columns):
 #   Column        Non-Null Count  Dtype
---  ------        --------------  -----
 0   Temperature   2500 non-null   float64
 1   Humidity      2500 non-null   float64
 2   Wind_Speed    2500 non-null   float64
 3   Cloud_Cover   2500 non-null   float64
 4   Pressure      2500 non-null   datetime64[ns]
 5   Rain          2500 non-null   object
 6   Rain_Encoded  2500 non-null   int64
dtypes: datetime64[ns](1), float64(4), int64(1), object(1)
memory usage: 136.8+ KB
None
```

	Temperature	Humidity	Wind_Speed	Cloud_Cover	Rain_Encoded
0	23.720338	89.592641	7.335604	50.501694	1
1	27.879734	46.489704	5.952484	4.990053	0
2	25.069084	83.072843	1.371992	14.855784	0
3	23.622080	74.367758	7.050551	67.255282	1
4	20.591370	96.858822	4.643921	47.676444	0

```
Processed Dataset Info:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 2500 entries, 0 to 2499
Data columns (total 5 columns):
 #   Column        Non-Null Count  Dtype
---  ------        --------------  -----
 0   Temperature   2500 non-null   float64
 1   Humidity      2500 non-null   float64
 2   Wind_Speed    2500 non-null   float64
 3   Cloud_Cover   2500 non-null   float64
 4   Rain_Encoded  2500 non-null   int64
dtypes: float64(4), int64(1)
memory usage: 97.8 KB
None
```

Task 12 – Movie Ratings Dataset Preparation

Task:

Clean and preprocess a movie ratings dataset using AI-generated scripts.

Instructions:

- Handle missing values in rating, genre.
- Encode categorical variables like genre, language.
- Remove duplicate movie entries.
- Scale rating values between 0 and 1.

Dataset link:

https://www.kaggle.com/datasets/PromptCloudHQ/imdb-data

Expected Output:

- A dataset suitable for recommendation systems.

Prompt : 1 Handle the missing values in rating, genre.

```
print("Missing values before handling:")
print(df[['Rating', 'Genre']].isnull().sum())
```

```
Missing values before handling:
Rating    0
Genre     0
dtype: int64
```

Prompt : 2 Encode the categorical variables like genre, language.

	Rank	Title	Genre	Description	Director	Actors	Yea
0	1	Guardians of the Galaxy	Action,Adventure,Sci-Fi	A group of intergalactic criminals are forced ...	James Gunn	Chris Pratt, Vin Diesel, Bradley Cooper, Zoe S...	201
1	2	Prometheus	Adventure,Mystery,Sci-Fi	Following clues to the origin of mankind, a te...	Ridley Scott	Noomi Rapace, Logan Marshall-Green, Michael Fa...	201
2	3	Split	Horror,Thriller	Three girls are kidnapped by a man with a diag...	M. Night Shyamalan	James McAvoy, Anya Taylor-Joy, Haley Lu Richar...	201
3	4	Sing	Animation,Comedy,Family	In a city of humanoid animals, a hustling thea...	Christophe Lourdelet	Matthew McConaughey,Reese Witherspoon, Seth Ma...	201
				A secret government		Will Smith, Jared	

Prompt : 3 Remove duplicate movie entries

```
[4]
0s
print(f"DataFrame shape before removing duplicates: {df.shape}")

df.drop_duplicates(inplace=True)

print(f"DataFrame shape after removing duplicates: {df.shape}")

DataFrame shape before removing duplicates: (1000, 32)
DataFrame shape after removing duplicates: (1000, 32)
```

Prompt : 4 Scale the rating values between 0 and 1.

```python
from sklearn.preprocessing import MinMaxScaler

# Initialize MinMaxScaler
scaler = MinMaxScaler()

# Reshape the 'Rating' column for scaling (MinMaxScaler expects a 2D array)
df['Rating_scaled'] = scaler.fit_transform(df[['Rating']])

# Display the DataFrame with the new scaled rating column
display(df[['Rating', 'Rating_scaled']].head())
```

	Rating	Rating_scaled
0	8.1	0.873239
1	7.0	0.718310
2	7.3	0.760563
3	7.2	0.746479
4	6.2	0.605634